

To allow multiple developers or users in your project to access the same database, you need to move away from using a local-only setup (like XAMPP/WAMP pointing to 127.0.0.1 or localhost) and host it on a centralized server.

Here are the standard approaches to centralize your **MySQL/MariaDB database**, ranked from the easiest to the most professional.

Option 1: Use a Cloud Database Service (Recommended)

Instead of hosting the database on your own computer, you host it with a cloud provider. This guarantees the database is always online and accessible via a standard connection string.

- **Free/Low-Cost Cloud Providers:** Railway, Render, PlanetScale, or Supabase (if you are open to PostgreSQL).
- **Major Cloud Providers:** AWS (RDS), Google Cloud SQL, or DigitalOcean Managed Databases.
- **How it works:** You create an account, spin up a MySQL instance, and the provider gives you a public hostname, username, password, and port. Every user in your project updates their local code configuration to point to this remote hostname instead of localhost.

Option 2: Share Your Local Machine (Quick for Hackathons/Testing)

If you just want to quickly share the database shown in your screenshot with teammates for a short coding session, you can expose your local port.

- **Using a Tunneling Tool (ngrok):**
 1. Download ngrok.
 2. Run the command: `ngrok tcp 3306` (assuming your MySQL runs on default port 3306).
 3. Ngrok will give you a public URL and port (e.g., `tcp://0.tcp.ngrok.io:12345`).
 4. Your teammates can connect to `0.tcp.ngrok.io` on port 12345.
- **Using Local Network (LAN):**

If your teammates are on the same Wi-Fi network, they can connect directly using your computer's local IP address (e.g., `192.168.1.X`) instead of localhost.

Option 3: Traditional Web Hosting (cPanel)

If your project already has a standard shared web hosting plan (like Hostinger, Bluehost, Namecheap, etc.):

1. Log into your hosting **cPanel**.
2. Go to **MySQL Databases** and create your database and user.
3. Look for an option called **Remote MySQL** or **Access Hosts**.
4. Add % (wildcard for any IP) or your teammates' specific public IP addresses to authorize

their connection.

Crucial Step: Update phpMyAdmin User Privileges

By default, local database installations restrict access to security-sensitive operations from outside connections. If you choose **Option 2** or **Option 3**, you must configure MySQL to allow remote logins:

1. In your **phpMyAdmin** dashboard, click on the **Privileges** tab at the top.
2. Look for your database user (e.g., root).
3. Ensure the **Host name** is set to % (which means "any host") rather than localhost or 127.0.0.1.
4. If it says localhost, edit the privileges or create a new user specifically for your team, setting their hostname to %.

To help narrow down the best setup for your team, could you tell me:

- What **programming language/framework** is your project using?
- Are your teammates working in the **same room/Wi-Fi**, or are they **remote**?
- Is this for a **temporary school project** or a **long-term production application**?